MALLORN Astronomical Classification Challenge

Machine Learning Final Project

Group 03

1. Introduction

This project addresses the MALLORN Astronomical Classification Challenge, a Kaggle competition aimed at identifying Tidal Disruption Events (TDEs) — rare astronomical phenomena where stars are torn apart by black holes — among 16 different types of astronomical objects. To tackle this binary classification challenge, we implemented and compared five machine learning approaches: Logistic Regression, K-Nearest Neighbors (KNN), Simple Neural Networks, Random Forest, and Support Vector Machines (SVM). Our methodology involved comprehensive feature engineering to extract statistical properties from light curves, various techniques to handle class imbalance (including threshold tuning, class weighting, SMOTE, and ensemble methods). Through extensive experimentation with different model configurations and validation strategies, we achieved our best Kaggle score of 0.4421 using a tuned SVM model, demonstrating that simpler, well-optimized models can outperform more complex ensemble approaches on this challenging astronomical classification task.

2. Dataset Description

The dataset consists of 3043 labeled astronomical objects with significant class imbalance, where TDEs represent only 148 samples compared to 1786 AGN (Active Galactic Nuclei) objects. Each object is characterized by metadata including redshift (Z), extinction coefficient (EBV), and light curve observations across six different filters (`u, g, r, i, z, y`) captured over time.

3. Preprocessing & Feature Engineering

3.1. Preprocessing

3.1.1. Light Curve Loading

For each astronomical object, we load its multi-band light curves from preprocessed CSV files. The data are split into training and testing sets, and all observations corresponding to the same `object_id` are aggregated to ensure correct alignment across photometric filters.

3.1.2. Feature Extraction

To enable object-level classification, each light curve is converted into a fixed-length feature vector consisting of **astrophysical metadata** and **statistical features**. Global properties provided in the log file, including redshift (`z`), galactic extinction (`ebv`), and redshift uncertainty (`z_err`) when available, are incorporated.

For each of the six photometric filters (`u, g, r, i, z, y`), filter-specific statistics are computed. To maintain a consistent feature dimension, filters without observations are filled with zeros. The extracted features include the number of observations; the mean, standard deviation, median, maximum, and minimum flux; flux skewness and range; the observation time span in Modified Julian Date (MJD); and the coefficient of variation of the flux.

3.1.3. Global Features

Additional global statistics across all filters are computed:

- Total number of observations
- Overall temporal baseline
- Global mean, standard deviation, maximum, and minimum flux

3.1.4. Dataset Construction

Feature extraction is applied to all objects in both datasets. For training samples, class labels (`target`) are appended. Each row in the final tabular dataset represents a single object.

```python
# preprocessing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train = train_features.drop(['object_id', 'target'], axis=1)
y_train = train_features['target']
X_test = test_features.drop(['object_id'], axis=1)

X_train = X_train.replace([np.inf, -np.inf], np.nan).fillna(0)
X_test = X_test.replace([np.inf, -np.inf], np.nan).fillna(0)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_scaled, y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)

print(f"\n訓練集: {X_tr.shape}, 驗證集: {X_val.shape}")
```

Figure 1. Code of Preprocessing

3.2. Feature Engineering

3.2.1. Feature Matrix and Normalization

Object identifiers and labels are removed from the input matrix. Infinite values are replaced with missing values and filled with zeros. Features are standardized using z-score normalization based on the training set, and the same transformation is applied to the test set.

3.2.2. Train–Validation Split

The training data are split into training and validation sets using an 80/20 stratified split to preserve class distribution. The validation set is used for model evaluation and hyperparameter tuning.

4. Model Implementation & Comparison

4.1. Simple-NN

We conducted four Simple-NN-based experiments with 2 enhanced strategies. One is dropout and the other is class weight.

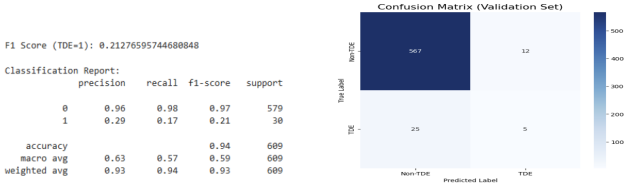
4.1.1. Experiment 1: Baseline

Implementation

In the first experiment, we implemented a basic fully connected layer MLP with uniform weight for each class as our baseline.

Results

Despite the high accuracy of the TDE class, the baseline Simple-NN model is ineffective for TDE detection due to extreme class imbalance. A large proportion of the data are concentrated in the negative–false region of the confusion matrix, and the low precision and recall for the Non-TDE class indicate that the predictions are overwhelmed by the Non-TDE class.



(a) F1 score (b) confusion matrix

Figure 2. Result of baseline

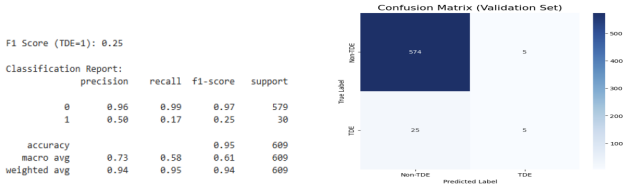
4.1.2. Experiment 2: baseline-NN + dropout

Implementation

To prevent the model from overfitting, we add a regularization method, dropout.

Results

Even though adding dropout to the baseline improves the F1 score in the validation and test sets, the Kaggle score decreases. Because the dataset is small and highly imbalanced, dropout removes too much informative signal from the TDE class, leading to underfitting instead of preventing overfitting.



(a) F1 score (b) confusion matrix

Figure 3. Result of baseline + dropout

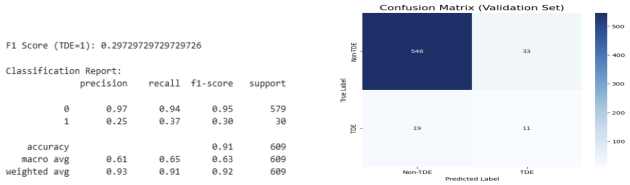
4.1.3. Experiment 3: baseline-NN + dropout + class weight

Implementation

In the third experiment, we add class weight to amplify the loss contribution of rare samples, namely, the TDE class.

Results

This improvement further increases the F1 score and achieves a higher Kaggle score compared to the previous experiment, since class weight encourages the model to learn more about the minority class (TDE) and prevents it from predicting only the majority class (Non_TDE).



(a) F1 score (b) confusion matrix

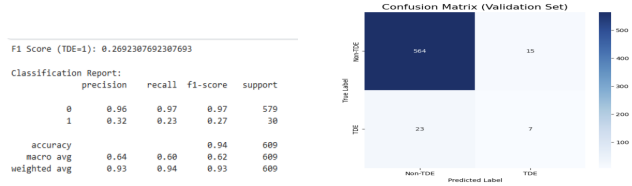
Figure 4. Result of baseline + dropout + class weight

4.1.4. Experiment 4: baseline-NN + class weight

Implementation

In the fourth experiment, we remove the dropout from the previous version to examine whether the dropout helps the model make accurate predictions or, instead, causes underfitting, as observed in the second experiment.

Results



(a) F1 score (b) confusion matrix

Figure 5. Result of baseline + class weight

Compared to the third experiment, both the F1 score and the Kaggle score decrease, indicating that dropout in the previous model serves its intended role in preventing overfitting. This is because class weight helps the model learn meaningful TDE patterns, reducing the dominance of the majority class in the final predictions on the imbalanced dataset.

4.1.5. Analysis of Simple-NN

Figure 6. Kaggle score of 4 experiments

After conducting the four experiments above, we observe a clear performance progression. The baseline-NN suffers from a highly imbalanced class resulting in poor detection of TDE. Although introducing dropout improves the F1 score on the validation and test sets, it leads to a lower Kaggle score because too much informative signal is removed.In contrast, applying class weight significantly improves model performance by forcing the model to focus more on the minority class, effectively mitigating the dominance caused by the imbalanced dataset. When dropout is added together with class weight, the previously mentioned issue no longer occurs. This is because class imbalance has already been alleviated by class weight. Under this more balanced learning condition, dropout becomes effective again, as it helps prevent overfitting and improves generalization performance.

4.2. KNN

To handle the severe class imbalance in the TDE classification task, we conduct a series of KNN-based experiments to evaluate the impact of different strategies on minority-class detection.

4.2.1. Experiment 1: Baseline KNN

Implementation

A baseline KNN classifier is trained using the standardized feature set. Hyperparameters, including the number of neighbors and distance weighting, are optimized via grid search with 5-fold cross-validation. The F1-score is used as the selection metric to emphasize performance on the minority TDE class.

The optimal configuration is:

- Number of neighbors: $K = 3$
- Weighting scheme: uniform

The model is trained on the original imbalanced dataset without resampling.

Results

Although the baseline KNN achieves high overall accuracy, it performs poorly on TDE detection. The validation F1-score is low, and the TDE recall is approximately 10%, indicating a strong bias toward the majority class. The confusion matrix reveals a large number of false negatives for TDE events.

This limitation is further reflected in the Kaggle leaderboard score of **0.1304**, demonstrating that accuracy is insufficient for evaluating rare-event classification under severe class imbalance.

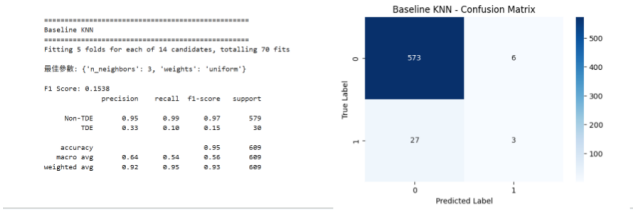


Figure 7. Baseline KNN result

Figure 8. Kaggle score of Baseline KNN

4.2.2. Experiment 2: KNN with SMOTE

Implementation

To mitigate class imbalance, we applied Synthetic Minority Over-sampling Technique (SMOTE) to the training set. SMOTE generated synthetic samples for the minority TDE class, resulting in a fully balanced dataset. The KNN classifier was retrained using the same hyperparameters obtained from Experiment 1 to isolate the effect of data balancing.

Results

Applying SMOTE significantly improved the model's ability to detect TDEs. The recall for the TDE class increased substantially, and the F1-score nearly doubled compared to the baseline. However, this improvement came at the cost of increased false positives, as reflected by reduced precision for the TDE class. The confusion matrix shows a more balanced prediction behavior, demonstrating that oversampling effectively reduced the majority-class bias.

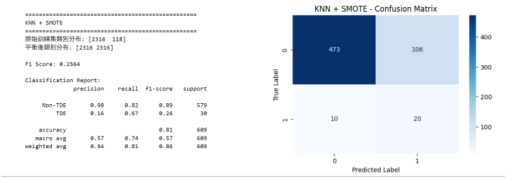


Figure 9. Result of KNN with SMOTE

Figure 10. Kaggle score of KNN with SMOTE

4.2.3. Experiment 3: KNN + SMOTE + XGBoost

Implementation

In the third experiment, we further enhanced the model by combining KNN with an XGBoost classifier using a soft-voting ensemble. The motivation was to leverage KNN's strength in local pattern recognition and XGBoost's ability to capture global decision boundaries. Both models were trained on the SMOTE-balanced dataset. The ensemble employed soft voting with a higher weight assigned to XGBoost (weight ratio 1:3).

Results

The ensemble model achieved the best overall performance among all experiments. The TDE class F1-score increased significantly, with both precision and recall reaching a more balanced level. The confusion matrix indicates a clear reduction in false negatives while maintaining strong performance on the Non-TDE class. This confirms that combining complementary models can effectively improve minority-class detection without excessively sacrificing majority-class accuracy.

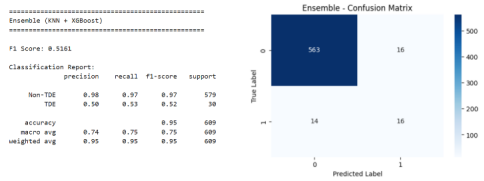


Figure 11. Result of KNN+XGBoost

Figure 12. Kaggle score of KNN+XGBoost

4.2.4. Analysis of KNN

Across the three experiments, a clear performance progression is observed. The baseline KNN suffered from severe class imbalance and failed to detect TDEs reliably. Introducing SMOTE substantially improved recall but introduced additional false positives. The ensemble approach successfully balanced this trade-off by improving recall while maintaining reasonable precision.

Overall, the results demonstrate that **data balancing is essential but insufficient on its own**, and that **ensemble learning provides a robust solution** for rare-event classification in astronomical datasets.

4.3. Random Forest

We implemented three Random Forest approaches with varying complexity: a single model with threshold tuning, an ensemble combining different tree-based models, and an ensemble using multiple Random Forest models with different random seeds.

4.3.1. Experiment 1: Random Forest without Ensemble

Implementation

We trained a single Random Forest model and explored different decision thresholds (0.2, 0.3, 0.4) to find the optimal threshold.

Results

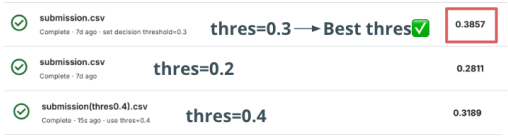


Figure 13. Kaggle score of different thresholds

The optimal threshold was 0.3, achieving F1=0.2857 on validation set with 10 true positives and 20 false negatives.

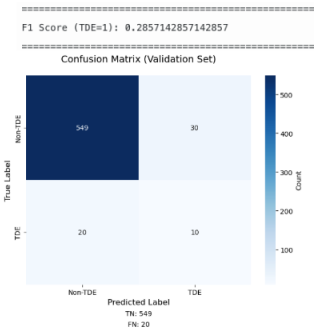


Figure 14. Results of threshold=0.3

4.3.2. Experiment 2: Ensemble Method with Different Models

Implementation

We combined three different tree-based models: Random Forest, XGBoost, and LightGBM using soft voting with threshold=0.3. Each model was trained independently and their predictions were averaged to produce the final classification.

Results

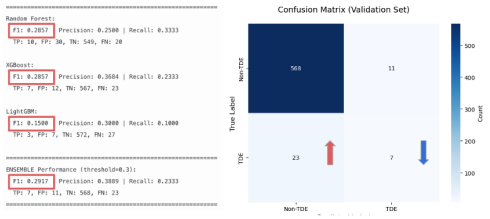


Figure 15. Result of ensemble with 3 tree-based models

The ensemble achieved higher F1 score on validation set (F1=0.2917) compared to single Random Forest. However, it resulted in more false negatives (23 vs 20), suggesting the ensemble approach sacrificed recall for precision.

4.3.3. Experiment 3: Ensemble with Different Seeds (same model)

Implementation

We trained multiple Random Forest models with different random seeds (seed=42, 123, 456) using the same hyperparameters. The predictions were aggregated using soft voting with threshold=0.3.

Results

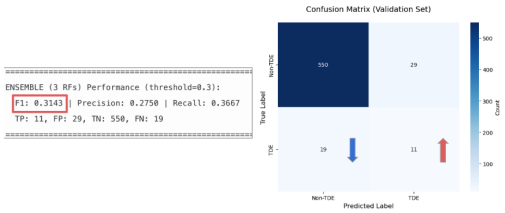


Figure 16. Result of ensemble with different seeds

This approach achieved the highest validation F1 score (F1=0.3143) among all Random Forest variants, with improved recall (36.67%) and fewer false negatives (19).

4.3.4. Analysis of Random Forest

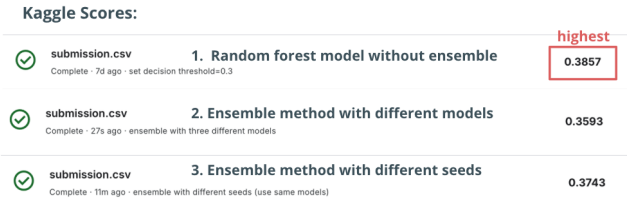


Figure 17. Kaggle score of different approaches

Interestingly, the baseline Random Forest achieved the highest test score of 0.3857, outperforming both the multi-model ensemble (0.3593) and multi-seed ensemble (0.3743). This contradicts validation results where the multi-seed ensemble showed the best F1 score of 0.3143. The superior performance of the simpler baseline on unseen data suggests that complex ensemble approaches may have overfit to validation characteristics. This highlights an important point: higher validation metrics do not guarantee better generalization, and simpler models with appropriate threshold tuning can be more robust on truly unseen data.

4.4. SVM

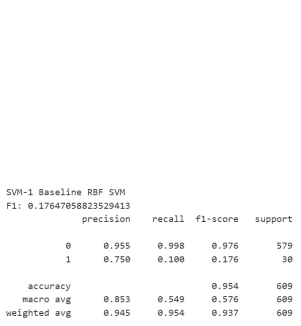
We implemented three SVM approaches with increasing complexity: a baseline RBF SVM using basic light-curve features, a class-weighted and hyperparameter-tuned SVM, and a PCA-based ensemble that averages the outputs of multiple SVMs trained with different random seeds.

4.4.1. Experiment 1: SVM with basic feature engineering

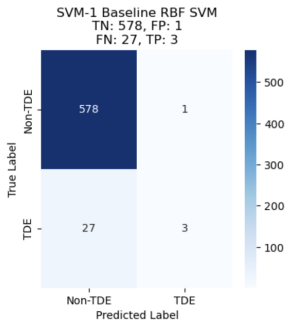
Implementation

We trained a baseline SVM only on the engineered light–curve features. We used a pipeline consisting of a StandardScaler followed by an RBF–kernel SVM (SVC with kernel="rbf"). All hyperparameters were kept at their scikit–learn defaults (C=10, gamma="scale", class_weight=None).

Results



(a) Experiment 1: F1 score



(b) Experiment 1: confusion matrix

Figure 18. Result of SVM baseline

The baseline SVM achieves an F1 score of about 0.18 for the TDE class. As shown in Figure 18b, the model mostly predicts "non–TDE": the recall for TDE is only 0.10 This shows the model is heavily biased toward the majority class under such an imbalanced setting.

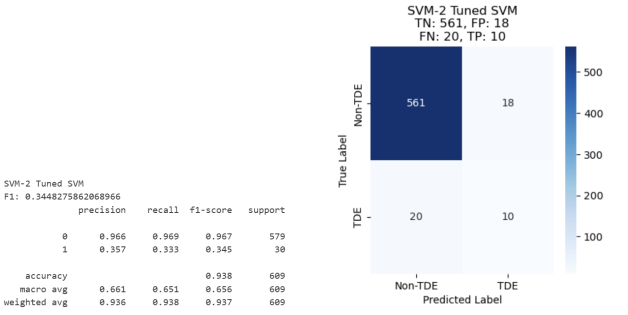
4.4.2. Experiment 2: SVM + class weight

Implementation

To encourage the model to pay more attention to TDEs, we introduced class weighting and tuned the hyperparameters using GridSearchCV. We searched over C $\in \{1, 3, 10, 30\}$, gamma $\in \{"scale", 0.01, 0.03, 0.1\}$, and class_weight $\in \{None, \{0:1, 1:3\}, \{0:1, 1:5\}\}$. The

grid search used 3–fold stratified cross–validation and directly optimized the F1 score of the TDE class.

Results



(a) Experiment 2: F1 score (b) Experiment 2: confusion matrix

Figure 19. Result of SVM + class weight

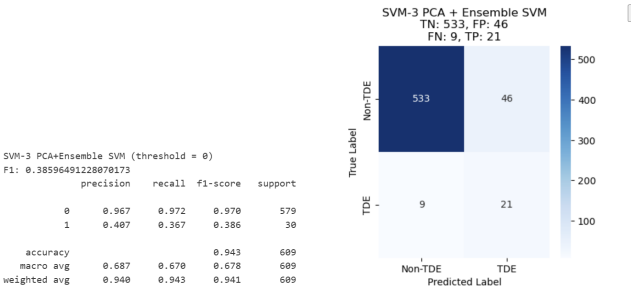
The best configuration found by the grid search is C = 3, gamma = "scale", and class_weight = {0:1, 1:5}. , and the TDE F1 score on the validation set improves to about 0.34. Compared to Experiment 1, the model now correctly recovers more TDEs (recall increases to 0.33), but also more false positives.

4.4.3. Experiment 3: SVM + PCA / Ensemble

Implementation

In this experiment we tried to combine PCA and model ensembling. we fixed the hyperparameters to the result from Experiment 2 and built a pipeline StandardScaler → PCA($n\_components = 50$) → SVM. Using this pipeline, we trained five models with different random seeds (0, 1, 2, 3, 4). we obtained the decision scores via decision_function, and averaged them to get the ensemble score.

Results



(a) Experiment 3: F1 score (b) Experiment 3: confusion matrix

Figure 20. Result of SVM + PCA / Ensemble

The ensemble achieves a TDE F1 score of about 0.39 and Compared to Experiment 2, recall on TDEs rises to 0.70, but the number of false positives also grows significantly.

4.4.4. Analysis of SVM

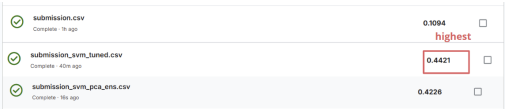


Figure 21. Kaggle scores of different approaches, showing that the tuned SVM (Experiment 2) generalizes best.

Across the three SVM experiments, performance improves step by step.The baseline RBF SVM has high accuracy but almost always predicts non-TDE, with very poor TDE recall and the worst Kaggle score.Adding class weights and tuning C, gamma (Experiment 2) greatly increases TDE recall and F1, and achieves the best Kaggle score (0.4421).PCA + ensemble (Experiment 3) slightly improves validation F1 but hurts Kaggle performance, suggesting extra complexity does not generalize as well as the tuned class-weighted SVM.

4.5. Logistic Regression

We conducted three logistic regression-based experiments. Two of them are related to decision boundary tuning. And one about ensemble learning.

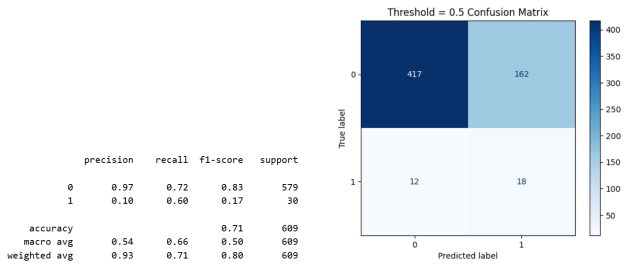
4.5.1. Experiment 1: Logistic Regression with Decision Boundary = 0.5

Implementation

In the first experiment, we used the LogisticRegression classifier from sklearn library to perform the classification task and simply set the decision boundary to 0.5.

Results

Using the default decision boundary of 0.5 is not suitable for highly imbalanced data like ours. Because the majority of samples are non-TDE objects, the model is biased toward predicting the majority class.



(a) Experiment 1: F1 score (b) Experiment 1: confusion matrix

Figure 22. Result of Logistic Regression with Decision Boundary = 0.5

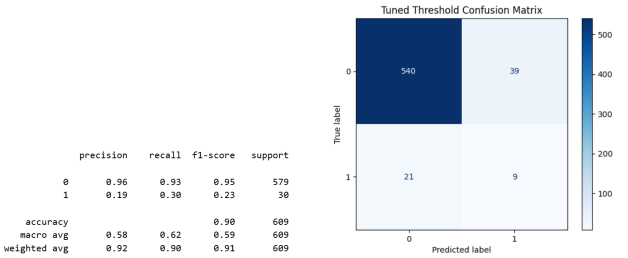
4.5.2. Experiment 2: Logistic Regression with Decision Boundary Tuning

Implementation

In the second experiment, instead of simply set the decision boundary to 0.5, we tried every possible threshold and choose the one with the highest score on the validation set as the final decision boundary.

Results

Generally, when encountering imbalanced data, thresholding tuning helps enhancing the recall for rare cases. However, to our suprise, the best threshold with the highest F1 score is 0.77, which is greater than 0.5. Even though we failed to catch more TDE events, we did avoid false positive cases.



(a) Experiment 2: F1 score (b) Experiment 2: confusion matrix

Figure 23. Result of Logistic Regression with Decision Boundary Tuning

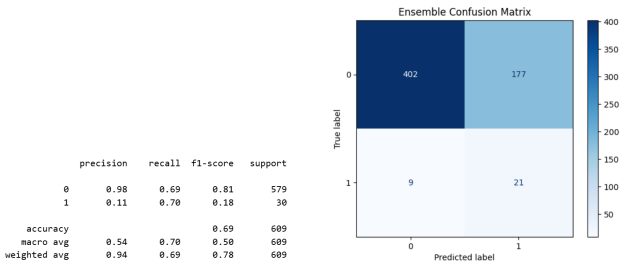
4.5.3. Experiment 3: Ensemble learning

Implementation

In the last experiment, we trained 20 models by giving each of them all the TDE cases and randomly choosing the same number of non-TDE cases. Apply voting before generating prediction. This kind of data arrangement might solve the dataset imbalanced issue.

Results

Letting models see all the TDE cases helps to improve recall. about 2/3 TDE cases can be detected. However, the overall f1 score didn't improve. Possible reason might be the model amplify validation-specific patterns.



(a) Experiment 3: F1 score (b) Experiment 3: confusion matrix

Figure 24. Result of ensemble learning

5. Results & Discussion

5.1. Strengths and Weaknesses

	Strengths	Weakness
Logistic Regression	simple, fast	Only capture linear relationships
KNN	can capture complex, non-linear patterns	sensitive to class imbalance
Simple NN	flexible, can learn complex non-linear mappings	requires large data, prone to overfitting
Random Forest	robust to outliers, no scaling needed	high complexity, may overfit
SVM	good in high dimensions, handles non-linearity	slow training, hyperparameter sensitive

5.2. Why Choose These Models

We selected five machine learning methods to systematically evaluate different learning paradigms for TDE classification under severe class imbalance and limited training data.

Baseline Models: Logistic Regression provides a simple linear classifier with high interpretability, while KNN offers a non-parametric approach capturing local patterns without distributional assumptions.

Advanced Models: We explored three advanced methods representing distinct paradigms: Simple Neural Networks (deep learning with automatic feature extraction), Random Forest (tree-based ensemble handling non-linear relationships), and Support Vector Machines (kernel-based methods for complex decision boundaries).

The diverse selection allows us to compare fundamentally different learning strategies and identify which approach best handles rare astronomical event detection with imbalanced, high-dimensional light-curve features.

5.3. Which method works best and why?

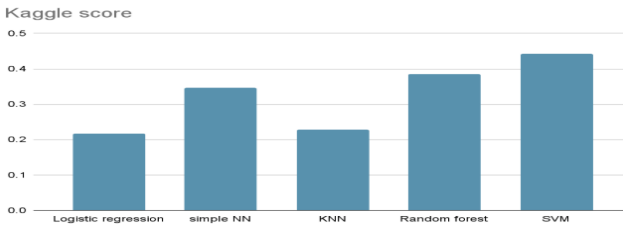


Figure 25. Kaggle score comparison of 5 models

From the Kaggle score comparison, SVM achieves the best performance (0.4421), followed by Random Forest

(0.3857) and Simple-NN (0.3466), while Logistic Regression (0.2178) and KNN without ensemble (0.2277) perform relatively worse.

This result can be explained by the characteristics of each model and how they interact with an imbalanced dataset.

5.3.1. Logistic Regression

Logistic Regression has the lowest performance because it is a linear classifier which assumes a linear decision boundary between classes. Therefore, underfitting happens.

5.3.2. KNN without ensemble

KNN also shows poor performance since it relies on local distance-based voting, which makes it vulnerable to imbalance. Although KNN can model nonlinear patterns, its local nature makes it unsuitable for imbalanced classification tasks, leading to poor performance.

5.3.3. Simple-NN

Simple-NN achieves moderate performance by introducing nonlinearity. Since Sensitive to class imbalance, applying class weighting can significantly improve performance. Thus, while Simple NN performs better than linear models, it does not fully exploit the structure of the data.

5.3.4. Random Forest

Random Forest achieves strong performance by combining multiple decision trees into an ensemble. Naturally handles non-linearity and feature interactions and robust to noise and outliers through ensemble averaging. Class imbalance still matters, but can be alleviated using class weights or balanced sampling. By aggregating diverse decision boundaries, Random Forest achieves strong generalization performance on this dataset.

5.3.5. SVM

SVM achieves the highest score due to several advantages:

- Effective handling of non-linearity through kernel functions
- Margin maximization: Improve generalization, especially in imbalanced settings.
- Handle class imbalance via class-weighted loss.

By focusing on support vectors near the decision boundary, SVM avoids being dominated by the majority class and learns a more discriminative boundary for minority samples.

5.3.6. Overall Conclusion

Among all tested methods, SVM performs best in this imbalanced dataset. This is because SVM can effectively model nonlinear decision boundaries while maintaining strong generalization through margin maximization. While other methods that are either too simple or too data-dependent. In addition, class-weighted SVM effectively mitigates class imbalance, allowing the model to focus on hard-to-classify minority samples.

5.4. Relation to Dataset and Feature Engineering

Our dataset is medium-sized but extremely imbalanced, with TDEs being a small minority class. The engineered features compress each multi-band light curve into global statistics, per-filter variability, and simple color/shape descriptors. Kernel SVM and Random Forest match this setting well: they model non-linear boundaries in moderately high-dimensional tabular data and still work with limited samples. In contrast, Logistic Regression underfits the complex decision boundary, KNN is strongly biased by the majority class in high dimensions, and the Simple NN lacks enough data to fully exploit its capacity.

5.5. Why did some ensembles underperform?

Although ensembles usually reduce variance, our ensembles sometimes hurt Kaggle performance. The Random Forest and SVM ensembles were tuned based on a single validation split and decision threshold, so they tended to overfit that split. PCA in SVM-3 may also discard minority–class information, and averaging multiple models further smooths the decision boundary. As a result, validation F1 slightly increased but the models produced more false positives and generalized worse on the hidden Kaggle test set.

5.6. Possible Improvements

Future work could focus on better handling of class imbalance and temporal structure. On the data side, methods such as SMOTE+ENN, class-balanced loss, or focal loss may improve TDE recall without exploding false positives. On the model side, gradient-boosted trees (XGBoost/LightGBM), calibrated probability outputs, or stacking ensembles could be explored. Finally, using sequence models on the raw light curves (e.g., 1D CNN/RNN/Transformer) may capture richer temporal patterns than our hand-crafted summary features.

Ability to explain the choice of certain methods (e.g., XGBoost vs. AdaBoost
vs. Logistic Regression). Provide a simple introduction to the strengths and weaknesses of the selected models.
Explain the meaning of key hyperparameters (e.g., learning rate, epoch, batch size, loss function, regularization strength).
It is not necessary to test all parameter combinations, but you should be able to explain what would be affected if those parameters were adjusted.